

AIM

To write a C program to create a Binary Search Tree and perform insertion, deletion, and searching of nodes without using a separate minimum value function.

PROCEDURE

1. Start the program.
2. Define a structure for the binary tree node containing data, left pointer, and right pointer.
3. Create a function to allocate memory and create a new tree node.
4. Insert nodes into the Binary Search Tree by comparing values and placing them in the correct position.
5. Search for a given value in the tree by recursively comparing node values.
6. Delete a node from the tree:
 - If the node has no child, delete it directly.
 - If the node has one child, replace it with its child.
 - If the node has two children, replace the node with its inorder predecessor.
7. Display the tree contents using inorder traversal.
8. Stop the program.

CODE

```
/*
FILE NAME    : tree.c
AUTHOR       : Monish Kanna
PURPOSE      : To implement a Binary Search Tree and perform create, insert, search, and delete
                operations without using a separate minimum value function.
*/
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node *top = NULL;
int isStackEmpty()
{ return (top == NULL); }
int isStackFull(){
    struct Node *temp = (struct Node*)malloc(sizeof(struct Node));
    if (temp == NULL)
        return 1;
    free(temp);
    return 0;
}
void pushStack(int value){
    if (isStackFull()){
        printf("Stack Overflow\n");
        return; }
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("%d pushed into stack\n", value);}
```

```

void popStack()
{
    if (isStackEmpty()){
        printf("Stack Underflow\n");
        return;
    }
    struct Node *temp = top;
    printf("%d popped from stack\n", temp->data);
    top = top->next;
    free(temp);
}
struct Node *front = NULL;
struct Node *rear = NULL;
void pushQueue(int value)
{
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if (rear == NULL)
    { front = rear = newNode; }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }
    printf("%d inserted into queue\n", value);
}
void popQueue()
{
    if (front == NULL) {
        printf("Queue is Empty\n");
        return;
    }
    struct Node *temp = front;
    printf("%d removed from queue\n", temp->data);
    front = front->next;
    if (front == NULL)
        rear = NULL;
    free(temp);
}
int main()
{
    pushStack(10);
    pushStack(20);
    popStack();
    popStack();
    pushQueue(100);
    pushQueue(200);
    popQueue();
    popQueue();
    return 0; }

```

OUTPUT

```
PROBLEMS  TERMINAL  OUTPUT  PORTS

PS C:\AMI C PROGRAM> cd "c:\AMI C PROGRAM\Monish_kanna_week3_practice3\" ;
● Inorder traversal: 20 30 40 50 60 70 80
  40 found in the tree
  After deleting 50: 20 30 40 60 70 80
```

RESULT

The C program to implement Stack and Queue using Linked List was executed successfully. Stack operations (push, pop, empty and full check) and Queue operations (insertion and deletion) were performed correctly using linked list concepts.